Enrolment No./Seat No_____

GUJARAT TECHNOLOGICAL UNIVERSITY

BE - SEMESTER–VI (NEW) EXAMINATION – SUMMER 2024

Subject Code:3160713 Date:22-05-2024

Subject Name: Web Programming

Time: 10:30 AM TO 01:00 PM Total Marks:70

Instructions:

1. **Attempt all questions.**
2. **Make suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Simple and non-programmable scientific calculators are allowed.**

			MARKS
Q.1	**(a)**	Define following: 1. URL 2. HTTP 3. Web Server	**03**
	(b)	Write differences between client-side scripting and server-side scripting.	**04**
	(c)	What is cascading style sheet (CSS)? Compare inline, embedded and external style sheet with example.	**07**
Q.2	**(a)**	Differentiate GET and POST methods.	**03**
	(b)	Explain different design issues at the time of designing an effective website.	**04**
	(c)	Explain “class” and “id” in CSS with suitable example.	**07**
		OR	
	(c)	Explain usefulness of rowspan and colspan attributes in HTML Table. Also, Write HTML code to print the following table.	**07**

State of Health	Fasting Value	
	Minimum	Maximum
Healthy	70	100
Pre-Diabetes	101	126
Diabetes	More than 126	N/A

Q.3	**(a)**	Write a short note on error handling in JavaScript.	**03**
	(b)	Discuss callback in JavaScript with the help of appropriate example.	**04**
	(c)	Design Student registration form including rollno, name, password and mobile number using HTML. Also write JavaScript to validate following: • All Fields are compulsory. • Password length should be 6 to 8 characters long. • Mobile number must be of 10 digits only.	**07**
		OR	
Q.3	**(a)**	Explain different types of popup boxes in JavaScript.	**03**
	(b)	Write a short note on DOM (Document Object Model) in JavaScript.	**04**

	(c)	Write a java script to find whether entered number by user is prime or not.	**07**
Q.4	**(a)**	Explain ordered and unordered list in HTML with example.	**03**
	(b)	Explain types of loops in PHP.	**04**
	(c)	What are different types of arrays in PHP? Explain with example to process the arrays in PHP.	**07**
		OR	
Q.4	**(a)**	Write a note on Web services.	**03**
	(b)	Write a PHP code to demonstrate a calculator using switch case.	**04**
	(c)	Briefly discuss sessions and cookies in PHP using appropriate example.	**07**
Q.5	**(a)**	Write a short note on file handling in PHP.	**03**
	(b)	Write a note on AJAX.	**04**
	(c)	Design Employee form with Employee ID, Employee Name, Designation details with submit button using HTML. On clicking submit button, entered Employee details should be fetch and display on another webpage "EmpInfo.php".	**07**
		OR	
Q.5	**(a)**	Briefly discuss JSON.	**03**
	(b)	Write the differences between synchronous and asynchronous web programming.	**04**
	(c)	Discuss steps to connect database using PHP. Also, discuss any two database operations using proper example.	**07**

****

Q.1 (a) Define following:

1. URL 2. HTTP 3. Web Server

1. URL (Uniform Resource Locator)

A URL is the **address of a resource on the Internet**. It specifies the **location of a web resource** and the **method used to access it**. A URL includes components such as protocol, domain name, and path. It helps browsers locate and retrieve web pages, images, or files from a server.

---

2. HTTP (HyperText Transfer Protocol)

HTTP is a **communication protocol** used for transferring data between a **web browser and a web server**. It defines rules for requesting and delivering web resources like HTML pages, images, and videos. HTTP follows a **request–response model**, where the client sends a request and the server sends back a response.

---

3. Web Server

A web server is a **computer system or software** that stores, processes, and delivers web pages to users. It handles client requests using protocols like HTTP or HTTPS. When a user enters a URL in a browser, the web server responds by sending the requested web page or resource.

(b) Write differences between client-side scripting and server-side scripting.

Basis of Comparison	Client-Side Scripting	Server-Side Scripting
Definition	Scripts executed on the **user's browser**.	Scripts executed on the **web server**.
Execution Location	Runs on the **client machine**.	Runs on the **server machine**.
Purpose	Used to create **interactive and dynamic user interfaces**.	Used to **process data and generate responses**.
Server Interaction	Does not require server for execution after page load.	Requires server interaction for every request.
Security	Less secure because code is visible to users.	More secure as code is hidden from users.

Basis of Comparison	Client-Side Scripting	Server-Side Scripting
Performance	Faster execution since no server round-trip is required.	Slightly slower due to server processing and communication.
Data Access	Cannot directly access server databases.	Can directly access databases and server resources.
Browser Dependency	Depends on browser compatibility.	Independent of browser; runs on server.
Examples of Languages	JavaScript, VBScript.	PHP, ASP.NET, Python, Java (Servlets).
Common Uses	Form validation, animations, UI effects.	User authentication, database operations, file handling.

(c) What is cascading style sheet (CSS)? Compare inline, embedded and external style sheet with example.

(c) Cascading Style Sheet (CSS)

Definition

Cascading Style Sheet (CSS) is a **style language used to control the appearance and layout of HTML web pages**. It is used to apply styles such as colors, fonts, spacing, alignment, and positioning to web elements. CSS separates **content (HTML)** from **presentation (design)**, which makes web pages easier to design, maintain, and update. The term *cascading* means that styles can be applied from multiple sources and follow a priority order.

Types of CSS

There are **three types of CSS**:

1. Inline CSS
2. Embedded (Internal) CSS
3. External CSS

Comparison of Inline, Embedded, and External CSS

Basis	Inline CSS	Embedded (Internal) CSS	External CSS
Definition	CSS is written **inside the HTML tag** using the `style` attribute.	CSS is written inside the `<style>` tag in the `<head>` section.	CSS is written in a **separate .css file** and linked to HTML.
Scope	Applies to **single element only**.	Applies to a **single web page**.	Applies to **multiple web pages**.
Reusability	Not reusable.	Limited reusability.	Highly reusable.
Maintenance	Difficult to maintain.	Easier than inline but limited.	Very easy to maintain and update.
Priority	Highest priority.	Medium priority.	Lowest priority.
Use Case	Small changes or testing styles.	Page-specific styling.	Large websites and common design.

Examples

1. *Inline CSS Example*

```
<p style="color: red; font-size: 18px;">This is Inline CSS</p>
```

---

2. *Embedded (Internal) CSS Example*

```
<style>
p {
  color: blue;
  font-size: 18px;
}
</style>
```

---

3. *External CSS Example*

```
<link rel="stylesheet" href="style.css">
p {
  color: green;
  font-size: 18px;
}
```

Q.2 (a) Differentiate GET and POST methods.

Basis of Comparison	GET Method	POST Method
Definition	GET method is used to **request or retrieve data** from the server.	POST method is used to **send data** to the server.
Data Transmission	Data is sent through the **URL**.	Data is sent through the **HTTP request body**.
Visibility	Data is **visible in the browser address bar**.	Data is **not visible** in the URL.
Security	Less secure because data is exposed.	More secure as data is hidden from URL.
Data Size Limit	Limited data can be sent.	Large amount of data can be sent.
Caching	Requests can be **cached** by browser.	Requests are **not cached**.
Bookmarking	Can be bookmarked.	Cannot be bookmarked.
Use Case	Used for search queries and fetching data.	Used for form submission and sensitive data.
Server Impact	Does not change server state.	Can modify server data.

(b) Explain different design issues at the time of designing an effective website.

Designing an effective website requires careful consideration of several design issues to ensure the website is **user-friendly, attractive, accessible, and functional**. The following points explain the important design issues that should be considered:

---

1. Usability and User Experience

The website should be easy to use and understand for all users. Navigation must be simple, menus should be clear, and important information should be easily accessible. A good user experience ensures that visitors can complete tasks without confusion.

2. Navigation Structure

Navigation should be consistent and logical throughout the website. Users should be able to move between pages easily using menus, links, and breadcrumbs. Poor navigation can frustrate users and increase bounce rate.

---

3. Visual Design and Layout

A clean and balanced layout improves readability and appearance. Proper use of spacing, alignment, and grid systems helps present content clearly. Overcrowded designs should be avoided to maintain focus.

---

4. Color Scheme and Typography

Colors should be chosen carefully to match the website's purpose and brand. Text must be readable with suitable font size and style. Poor color contrast and inappropriate fonts can reduce readability.

---

5. Content Quality and Readability

Content should be relevant, well-structured, and easy to understand. Headings, paragraphs, and lists should be used properly. Poor or cluttered content reduces user engagement.

---

6. Responsiveness and Compatibility

The website should work well on different devices such as mobiles, tablets, and desktops. It must also be compatible with various browsers. Responsive design ensures a consistent experience across platforms.

---

7. Page Load Speed

Heavy images, unnecessary scripts, and large files slow down the website. Optimized images and clean code help improve loading speed, which is critical for user satisfaction and SEO.

---

8. Accessibility

The website should be accessible to users with disabilities. Proper use of alt text, readable fonts, and keyboard navigation improves accessibility. This ensures inclusiveness and legal compliance.

---

9. Security Considerations

User data must be protected using secure forms and HTTPS. Poor security design can lead to data breaches and loss of user trust.

---

10. Consistency

Design elements such as colors, fonts, buttons, and layout should remain consistent across all pages. Consistency improves usability and builds a professional appearance.

(c) Explain “class” and “id” in CSS with suitable example.

In **Cascading Style Sheets (CSS)**, **class** and **id** are selectors used to apply styles to HTML elements. They play an important role in controlling the design and layout of web pages by allowing developers to target specific elements efficiently. Although both are used for styling, their purpose and usage are different.

---

1. Class in CSS

Definition

A **class** is a CSS selector used to apply the **same style to multiple HTML elements**. It helps in grouping elements that share similar appearance or behavior. A class selector is written using a **dot (.)** followed by the class name.

Explanation

Classes are reusable and flexible, making them very useful in large websites. By assigning the same class name to different elements, we can maintain a consistent look across the webpage. If any change is required, it can be done in one place, which improves maintainability.

Features of Class

- Can be applied to **multiple elements**
- Reusable across the page
- Improves code readability and consistency

- Lower priority compared to id selector

Example

```
<p class="info">Student Information</p>
<p class="info">Course Details</p>
.info {
  color: green;
  font-size: 16px;
}
```

In this example, both paragraphs share the same style using a single class.

---

2. ID in CSS

Definition

An **id** is a CSS selector used to apply a style to a **single unique HTML element**. Each id must be unique within a webpage. The id selector is written using a **hash (#)** followed by the id name.

Explanation

IDs are generally used for unique sections such as headers, footers, navigation bars, or main content areas. Because an id is unique, it has a **higher priority** than class selectors, meaning its styles override class styles when both are applied.

Features of ID

- Applied to **only one element**
- Must be unique in the document
- Has higher priority than class
- Commonly used for layout and JavaScript targeting

Example

```
<h2 id="mainTitle">Website Design</h2>
#mainTitle {
  color: blue;
  text-align: center;
}
```

This example styles only one heading using an id.

Difference Between Class and ID

Basis	Class	ID
Purpose	Style multiple elements	Style a unique element
Symbol	. (dot)	# (hash)
Reusability	Can be reused	Cannot be reused
Priority	Lower	Higher
Common Use	Styling groups of elements	Styling unique sections

OR

(c) Explain usefulness of rowspan and colspan attributes in HTML Table. Also, Write HTML code to print the following table.

State of Health	Fasting Value	
	Minimum	Maximum
Healthy	70	100
Pre-Diabetes	101	126
Diabetes	More than 126	N/A

Usefulness of `rowspan` Attribute

Definition

The **`rowspan`** attribute in HTML tables is used to **merge two or more rows into a single cell vertically**.

Why it is useful

- It helps represent **common data shared across multiple rows**
- Reduces repetition of same content
- Improves **table readability and structure**
- Commonly used in headings, categories, and grouped data

Example Use Case

If one heading applies to multiple rows (like a main category), `rowspan` is used to display it once while covering multiple rows.

Usefulness of `colspan` Attribute

Definition

The **`colspan`** attribute is used to **merge two or more columns into a single cell horizontally**.

Why it is useful

- Useful for **group headings**
- Helps create complex table layouts
- Makes tables look **organized and professional**
- Avoids unnecessary extra columns

Example Use Case

When a heading applies to multiple columns (like "Fasting Value" covering Minimum and Maximum), `colspan` is used.

Explanation of the Given Table Structure

In the given table:

- **"State of Health"** occupies **two rows** → uses `rowspan="2"`
- **"Fasting Value"** covers **two columns (Minimum and Maximum)** → uses `colspan="2"`
- Sub-headings **Minimum** and **Maximum** are under **Fasting Value**
- Data rows represent Healthy, Pre-Diabetes, and Diabetes conditions

---

HTML Code to Print the Given Table

```
<!DOCTYPE html>
<html>
<head>
  <title>Fasting Blood Sugar Table</title>
</head>
<body>

<table border="1" cellpadding="10" cellspacing="0">
  <tr>
    <th rowspan="2">State of Health</th>
    <th colspan="2">Fasting Value</th>
  </tr>
  <tr>
```

```
    <th>Minimum</th>
    <th>Maximum</th>
  </tr>
  <tr>
    <td>Healthy</td>
    <td>70</td>
    <td>100</td>
  </tr>
  <tr>
    <td>Pre-Diabetes</td>
    <td>101</td>
    <td>126</td>
  </tr>
  <tr>
    <td>Diabetes</td>
    <td>More than 126</td>
    <td>N/A</td>
  </tr>
</table>

</body>
</html>
```

Q.3 (a) Write a short note on error handling in JavaScript.

Error Handling in JavaScript

Error handling in JavaScript is the mechanism used to **detect, manage, and respond to runtime errors** so that a program does not crash unexpectedly and can continue execution smoothly. Proper error handling helps developers **debug issues, improve reliability, and provide meaningful feedback to users**.

---

1. Types of Errors in JavaScript

JavaScript errors are mainly classified into three types:

(a) Syntax Errors

- Occur due to incorrect syntax in the code.
- Detected at the time of parsing before execution.
- Example: missing brackets, invalid keywords.

(b) Runtime Errors

- Occur during program execution.
- Caused by invalid operations like accessing undefined variables or calling non-existing functions.

(c) Logical Errors

- Occur due to incorrect program logic.
- The program runs but produces incorrect output.
- Harder to detect because no error message is shown.

---

2. try…catch Statement

JavaScript provides the **try…catch** block to handle runtime errors.

- `try` block contains code that may cause an error.
- `catch` block executes when an error occurs.
- Prevents program termination due to errors.

Example:

```
try {
    let a = 10;
    let b = a / c;   // c is not defined
    console.log(b);
}
catch (error) {
    console.log("Error occurred:", error.message);
}
```

---

3. finally Block

- The **finally** block executes **regardless of whether an error occurs or not**.
- Used for cleanup tasks such as closing resources.

Example:

```
try {
    let x = 5;
    console.log(x);
}
catch (error) {
    console.log("Error occurred");
}
finally {
    console.log("Execution completed");
}
```

---

4. throw Statement

- The **throw** keyword is used to create **custom errors**.
- Helps in validating user input and handling application-specific issues.

Example:

```
let age = 15;

try {
    if (age < 18) {
        throw "Age must be 18 or above";
    }
    console.log("Access granted");
}
catch (error) {
    console.log("Error:", error);
}
```

---

5. Importance of Error Handling

- Prevents application crashes
- Improves user experience
- Helps in debugging and maintenance
- Ensures secure and stable applications

(b) Discuss callback in JavaScript with the help of appropriate example.

Callback in JavaScript

A **callback** in JavaScript is a **function that is passed as an argument to another function and is executed later**, after the completion of a specific task. Instead of returning a value immediately, the calling function "calls back" the passed function once its work is done. Callbacks are a core concept in JavaScript and form the foundation of **asynchronous programming**.

Need for Callbacks

JavaScript is a **single-threaded, non-blocking language**, which means it can execute only one operation at a time. To avoid stopping the entire program while waiting for time-consuming operations (such as timers, user actions, or server responses), JavaScript uses callbacks. Callbacks ensure that dependent code runs **only after** the required task has been completed.

How Callbacks Work

- A function is defined normally.

- Another function accepts this function as a parameter.
- After completing its main task, the function executes the callback.
- This allows proper execution order without blocking the program flow.

Example of Callback in JavaScript

```
function displayResult(result) {
    console.log("Result is:", result);
}

function addNumbers(a, b, callback) {
    let sum = a + b;
    callback(sum);
}

addNumbers(10, 20, displayResult);
```

Callbacks in Asynchronous Operations

Callbacks are widely used in asynchronous tasks such as:

- `setTimeout()` and `setInterval()`
- Event handling (click, keypress, etc.)
- Server requests and responses

They allow JavaScript to continue executing other code while waiting for the task to finish.

Advantages of Callbacks

- Enable asynchronous programming
- Maintain proper execution sequence
- Improve application responsiveness
- Essential for event-driven programming

Limitations of Callbacks

When callbacks are nested inside multiple callbacks, the code becomes complex and difficult to manage. This problem is known as **callback hell**, which affects readability and maintainability.

(c) Design Student registration form including rollno, name, password and mobile number using HTML. Also write JavaScript to validate following: • All

Fields are compulsory. • Password length should be 6 to 8 characters long. • Mobile number must be of 10 digits only.

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Registration Form</title>
</head>
<body>

<h2>Student Registration Form</h2>

<form onsubmit="return validateForm()">

   Roll No: <br>
   <input type="text" id="rollno"><br><br>

   Name: <br>
   <input type="text" id="name"><br><br>

   Password: <br>
   <input type="password" id="password"><br><br>

   Mobile Number: <br>
   <input type="text" id="mobile"><br><br>

   <input type="submit" value="Register">

</form>
```

```
<script>
function validateForm() {

   let rollno = document.getElementById("rollno").value;
   let name = document.getElementById("name").value;
   let password = document.getElementById("password").value;
   let mobile = document.getElementById("mobile").value;

   // Check all fields are compulsory
   if (rollno === "" || name === "" || password === "" || mobile === "") {
      alert("All fields are compulsory");
      return false;
   }

   // Password length validation
   if (password.length < 6 || password.length > 8) {
      alert("Password must be 6 to 8 characters long");
      return false;
   }

   // Mobile number validation
   if (mobile.length !== 10 || isNaN(mobile)) {
      alert("Mobile number must be exactly 10 digits");
      return false;
   }

   alert("Registration Successful");
   return true;
}
```

```
</script>


</body>
</html>
```

OUTPUT:

Student Registration Form

Roll No: [_____]

Name: [_____]

Password: [_____]

Mobile Number: [_____]

Register

OR

Q.3 (a) Explain different types of popup boxes in JavaScript.

Popup Boxes in JavaScript

JavaScript provides **built-in popup boxes** that allow interaction between the **webpage and the user**. They are simple **modal dialogs** that appear on the screen and require the user to respond before continuing. There are three main types: **Alert, Confirm, and Prompt**.

---

1. Alert Box

- **Purpose:** To display **information, warnings, or notifications** to the user.
- **Behavior:** Shows a **message** with a single **OK** button. The user cannot interact with the page until they click **OK**.
- **Return value:** `undefined` (just displays message).

Syntax

```
alert("Message goes here");
```

Example

```
alert("Registration Successful!");
```

Explanation:

- A small dialog box appears with the message `"Registration Successful!"`.
- User must click **OK** to continue using the page.
- Commonly used in form validation to show errors or success messages.

---

2. Confirm Box

- **Purpose:** To **ask the user to confirm or reject** an action.
- **Behavior:** Displays a message with **OK** and **Cancel** buttons.
- **Return value:**
  - `true` → if the user clicks **OK**
  - `false` → if the user clicks **Cancel**

Syntax

```
let result = confirm("Do you want to continue?");
```

Example

```
let deleteItem = confirm("Are you sure you want to delete this record?");
if(deleteItem) {
    console.log("Record deleted");
} else {
    console.log("Action canceled");
}
```

Explanation:

- Used when an action has **consequences**, like deleting data.
- User choice is captured in a variable (`result` or `deleteItem`) for further logic.
- Ensures that **accidental clicks are avoided**.

---

3. Prompt Box

- **Purpose:** To **collect input from the user**.
- **Behavior:** Displays a message with a **text input field** and **OK/Cancel** buttons.
- **Return value:**
  - The **text entered** → if user clicks **OK**
  - `null` → if user clicks **Cancel**

Syntax

```
let input = prompt("Enter your name:");
```

Example

```
let name = prompt("Please enter your name:");
if(name) {
    alert("Hello, " + name + "!");
} else {
    alert("You did not enter your name");
}
```

Explanation:

- Useful for **simple user input**, like asking a name, age, or code.
- Input can be stored in a variable and used in program logic.

(b) Write a short note on DOM (Document Object Model) in JavaScript.

The **Document Object Model (DOM)** is a **programming interface for web documents**. It represents an **HTML or XML document as a structured tree of objects**, allowing JavaScript to **interact with, manipulate, and modify the content, structure, and style** of a webpage dynamically. In simpler words, the DOM is a **bridge between JavaScript and HTML**, making web pages interactive.

---

1. Structure of DOM

- A web page is represented as a **tree of nodes**, starting from the **Document** node.
- Every HTML element, attribute, and piece of text becomes a **node** in this tree.
- Example of DOM tree for a simple HTML page:

```
Document
└── html
    ├── head
    │   └── title
    └── body
        ├── h1
        ├── p
        └── button
```

- **Nodes in DOM:**
    1. **Element nodes:** Represent HTML elements like `<div>`, `<p>`, `<h1>`.
    2. **Text nodes:** Contain the text inside elements.
    3. **Attribute nodes:** Represent attributes of elements like `id`, `class`, `src`.

---

2. Importance of DOM

- **Access and Manipulation:** Allows JavaScript to access elements and change their content, style, or attributes.
- **Dynamic Interaction:** Enables interaction with users without reloading the page.

- **Event Handling:** Lets developers respond to user actions like clicks, keypresses, or mouse movements.
- **Foundation for Modern Web Development:** Essential for building interactive and responsive web pages.

---

3. Common DOM Methods

- **Accessing elements:**
  - `document.getElementById("id")` – Access element by ID
  - `document.getElementsByClassName("class")` – Access elements by class
  - `document.getElementsByTagName("tag")` – Access elements by tag name
  - `document.querySelector()` / `document.querySelectorAll()` – Access elements using CSS selectors
- **Manipulating elements:**
  - `element.innerHTML` – Get or set the content of an element
  - `element.style` – Modify CSS styles dynamically
  - `element.value` – Get or set values of input fields

---

4. DOM Events

- Events are **user actions or browser events** that can trigger JavaScript functions.
- Examples: `click`, `mouseover`, `keydown`, `submit`.

Example:

```
document.getElementById("btn").onclick = function() {
    alert("Button clicked!");
}
```

- This code executes when the user clicks a button.

---

5. Simple Example of DOM Manipulation

```
<h1 id="heading">Hello World!</h1>
<button onclick="changeText()">Click Me</button>

<script>
function changeText() {
    document.getElementById("heading").innerHTML = "Welcome to JavaScript
DOM!";
}
</script>
```

Explanation:

- `document.getElementById("heading")` accesses the `<h1>` element.
- `innerHTML` changes the text when the button is clicked.
- Demonstrates how DOM allows **dynamic interaction** with a webpage.

---

6. Key Points

1. The DOM is **language-independent** but is used in JavaScript to control web pages.
2. It converts HTML into a **tree structure** of objects called nodes.
3. Using DOM, developers can **read, add, modify, or delete elements and content**.
4. DOM events allow websites to be **interactive and responsive**.

(c) Write a java script to find whether entered number by user is prime or not.

```
<!DOCTYPE html>

<html>

<head>

   <title>Prime Number Check</title>

</head>

<body>


<h2>Check Prime Number</h2>


<input type="number" id="num" placeholder="Enter a number">

<button onclick="checkPrime()">Check</button>


<p id="result"></p>


<script>

function checkPrime() {

   let number = parseInt(document.getElementById("num").value);

   let result = document.getElementById("result");
```

```
    if(number <= 1) {
        result.innerHTML = number + " is not a prime number.";
        return;
    }

    let isPrime = true;

    for(let i = 2; i <= Math.sqrt(number); i++) {
        if(number % i === 0) {
            isPrime = false;
            break;
        }
    }

    if(isPrime) {
        result.innerHTML = number + " is a prime number.";
    } else {
        result.innerHTML = number + " is not a prime number.";
    }
}
</script>

</body>
</html>
```

Input	Output
7	7 is a prime number.
12	12 is not a prime number.
1	1 is not a prime number.

Q.4 (a) Explain ordered and unordered list in HTML with example.

1. Ordered List (`<ol>`)

- An **ordered list** displays items **in a specific sequence**, usually numbered.
- Each list item is placed inside the `<li>` (list item) tag.
- Browsers **automatically number** the items.

Syntax

```
<ol>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

Example

```
<h3>Top 3 Programming Languages</h3>
<ol>
  <li>Python</li>
  <li>JavaScript</li>
  <li>Java</li>
</ol>
```

Output:

1. Python
2. JavaScript
3. Java

Key Points:

- Numbering is **automatic** (1, 2, 3, …).
- You can change numbering style using the `type` attribute (`1`, `A`, `a`, `I`, `i`).

---

2. Unordered List (`<ul>`)

- An **unordered list** displays items **without any specific order**, usually marked with **bullets**.
- Each list item is also placed inside the `<li>` tag.

Syntax

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

Example

```
<h3>Fruits I Like</h3>
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Mango</li>
</ul>
```

Output:

- Apple
- Banana
- Mango

Key Points:

- Default bullets can be changed using the `type` attribute (`disc`, `circle`, `square`) or CSS.
- Order of items **does not matter**.

(b) Explain types of loops in PHP.

Types of Loops in PHP

Loops are used to **execute a block of code repeatedly** until a certain condition is met. They are essential in PHP for **reducing code redundancy, handling repetitive tasks, and improving program efficiency**. PHP provides **four main types of loops**: `while`, `do…while`, `for`, and `foreach`.

---

1. While Loop

Definition:
The **while loop** is used to execute a block of code **as long as a specified condition remains true**. The condition is evaluated **before the loop begins**, so if the condition is initially false, the code inside the loop may **not execute at all**. It is mostly used when the **number of iterations is not known in advance**.

Key Points:

- Checks the condition **before executing the loop**.
- Can result in an **infinite loop** if the condition never becomes false.
- Ideal when the loop should **run until a certain condition changes**.

Example:

```
$i = 1;
while ($i <= 5) {
    echo "Number: $i <br>";
```

```
    $i++;
}
```

Output:
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5

---

2. Do…While Loop

Definition:
The **do…while loop** is similar to the while loop, but it guarantees that the **block of code runs at least once** because the condition is evaluated **after the loop has executed**. It is useful when you want the code to run once **before checking the condition**.

Key Points:

- Executes the code **at least once**.
- Checks the condition **after execution**.
- Can also create infinite loops if the condition is always true.

Example:

```
$i = 1;
do {
    echo "Number: $i <br>";
    $i++;
} while ($i <= 5);
```

Output: Same as above.

---

3. For Loop

Definition:
The **for loop** is used when the **number of iterations is known in advance**. It has an **initialization, condition, and increment/decrement**. This loop is compact and widely used in PHP for **repeating tasks a fixed number of times**.

Key Points:

- Initialization happens once at the start.
- Condition is checked before each iteration.
- Increment/decrement changes the counter after each iteration.
- Efficient for **count-controlled loops**.

Example:

```
for ($i = 1; $i <= 5; $i++) {
    echo "Number: $i <br>";
}
```

Output: Same as above.

---

4. Foreach Loop

Definition:
The **foreach loop** is specifically designed for **iterating over arrays**. It automatically retrieves each element, eliminating the need for a counter or index variable. It is the most convenient loop for working with **arrays and associative arrays**.

Key Points:

- Automatically iterates over each element of an array.
- Can access both **key and value** in associative arrays.
- Makes array processing **simpler and cleaner** than using a for loop.

Example:

```
$fruits = array("Apple", "Banana", "Mango");
foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}
```

Output:
Apple
Banana
Mango

Example with Key and Value:

```
$ages = array("Jiya" => 22, "Govind" => 25);
foreach ($ages as $name => $age) {
    echo "$name is $age years old<br>";
}
```

Output:
Jiya is 22 years old
Govind is 25 years old

(c) What are different types of arrays in PHP? Explain with example to process the arrays in PHP.

Arrays in PHP – Detailed Theory

An **array** in PHP is a **special variable** that allows storing **multiple values in a single variable**. Unlike normal variables that can hold only one value at a time, arrays can hold **a list or collection of values**, which can be **numbers, strings, or even other arrays**. Arrays are one of the most important data structures in PHP, as they allow **efficient storage, organization, and manipulation of data**.

Why Use Arrays in PHP?

- To **store multiple related values** together, e.g., list of students, products, or numbers.
- To **avoid creating many individual variables**, which makes code cleaner and easier to maintain.
- To **process data efficiently using loops**.
- To allow **mapping of keys to values** (associative arrays) for better data organization.
- To support **complex data structures** such as tables and matrices (multidimensional arrays).

Types of Arrays in PHP

PHP provides three main types of arrays, each suitable for different scenarios:

---

1. Indexed Array

Definition:
An **indexed array** is an array where **each element is assigned a numeric index** automatically, starting from 0. Indexed arrays are useful when the **sequence of data matters**, such as a list of items or numbers.

Key Points:

- Elements are accessed using **numeric indexes**.
- Indexes start from **0** by default.
- Can be processed using **for loops** or **foreach loops**.
- The size of the array can be determined using the `count()` function.

Example:

```
$fruits = array("Apple", "Banana", "Mango");
```

```
for ($i = 0; $i < count($fruits); $i++) {
    echo $fruits[$i] . "<br>";
}
```

Output:

```
Apple
Banana
Mango
```

---

2. Associative Array

Definition:
An **associative array** stores elements with **custom keys** instead of numeric indexes. Keys can be strings or numbers, which makes it possible to **map meaningful identifiers to values**. Associative arrays are commonly used to **store related data in key-value pairs**, such as names and ages, product IDs and names, etc.

Key Points:

- Keys can be **strings or integers**.
- Elements are accessed using their **key names**.
- Processing is usually done using **`foreach` loops** for simplicity.
- Makes code more **readable and organized**.

Example:

```
$ages = array("Krishna" => 22, "Nivan" => 25, "Ravi" => 20);

foreach ($ages as $name => $age) {
    echo "$name is $age years old<br>";
}
```

Output:

```
Krishna is 22 years old
Nivan is 25 years old
Ravi is 20 years old
```

---

3. Multidimensional Array

Definition:
A **multidimensional array** is an array that contains **one or more arrays as its elements**. This allows storing **complex data structures**, like tables, matrices, or records with multiple attributes. Multidimensional arrays are useful for representing **rows and columns** of data.

Key Points:

- Each element of the main array can be **another array**.
- Can be **nested to multiple levels** depending on complexity.
- Accessed using **multiple indexes**: `$array[row][column]`.
- Processed using **nested loops**.

Example:

```
$students = array(
    array("nian", 22, "B.Sc"),
    array("Raj", 25, "BCA"),
    array("Ravi", 20, "BBA")
);

for ($i = 0; $i < count($students); $i++) {
    for ($j = 0; $j < count($students[$i]); $j++) {
        echo $students[$i][$j] . " ";
    }
    echo "<br>";
}
```

Output:

```
nian 22 B.Sc
Raj 25 BCA
Ravi 20 BBA
```

Processing Arrays in PHP

Arrays in PHP can be **processed using loops** or **array functions**:

1. **Using Loops**:
    - **Indexed arrays:** `for` loop is commonly used.
    - **Associative arrays:** `foreach` loop is preferred.
    - **Multidimensional arrays:** Nested loops are used for rows and columns.
2. **Using Built-in Functions**:
    - `count($array)` – Returns the number of elements.
    - `array_push()` – Adds elements to the end.
    - `array_pop()` – Removes the last element.
    - `array_keys()` – Returns all keys.
    - `array_values()` – Returns all values.

OR

Q.4 (a) Write a note on Web services.

A **web service** is a **software system designed to support interoperable machine-to-machine communication over a network**, typically the Internet. In simpler terms, it allows **different applications or systems to communicate with each other**, regardless of the programming language, platform, or device they are built on.

Web services are **used to share data, functions, or services** between applications in a **standardized way**.

Key Features of Web Services

1. **Platform Independent:**
   - Web services can run on **any operating system** or platform.
   - Example: A PHP application can communicate with a Java-based system.
2. **Language Independent:**
   - Applications written in different programming languages can **interact seamlessly** using web services.
3. **Use of Standard Protocols:**
   - Web services use **HTTP, HTTPS, SOAP, REST, XML, JSON**, etc., to exchange information.
4. **Interoperability:**
   - Enables different systems and applications to **work together**.
5. **Reusability:**
   - Services can be reused in **different applications** without rewriting code.

Types of Web Services

1. **SOAP Web Services (Simple Object Access Protocol):**
   - Uses **XML-based messaging** protocol.
   - Standardized, supports **complex operations**, but heavier in data size.
2. **RESTful Web Services (Representational State Transfer):**
   - Uses **HTTP methods** like GET, POST, PUT, DELETE.
   - Lightweight, supports **JSON/XML**, widely used for web and mobile apps.

Working of Web Services

1. **Service Provider:** Creates and publishes the web service.
2. **Service Requestor/Client:** Sends a request to access the service.
3. **Service Registry (optional):** Helps clients find available services.
4. **Communication:** The service provider responds with data or executes the requested operation.

Example Scenario:

- A **weather app** fetches weather data from a web service provided by a **weather information server** using REST API.
- The app receives data in **JSON format** and displays it to users.

Advantages of Web Services

- Enables **remote communication between applications**.
- **Reduces development effort** by reusing existing services.
- Supports **integration** of heterogeneous systems.
- Provides **scalability and modularity** in application design.

(b) Write a PHP code to demonstrate a calculator using switch case.

```
<!DOCTYPE html>
<html>
<head>
  <title>PHP Calculator</title>
</head>
<body>
<h2>Simple PHP Calculator</h2>
<form method="post" action="">
  Enter First Number: <input type="number" name="num1" required><br><br>
  Enter Second Number: <input type="number" name="num2" required><br><br>
  Select Operation:
  <select name="operation" required>
    <option value="add">Addition (+)</option>
    <option value="subtract">Subtraction (-)</option>
    <option value="multiply">Multiplication (*)</option>
    <option value="divide">Division (/)</option>
  </select><br><br>
  <input type="submit" name="calculate" value="Calculate">
```

```
</form>

<?php
if(isset($_POST['calculate'])) {
    $num1 = $_POST['num1'];
    $num2 = $_POST['num2'];
    $operation = $_POST['operation'];
    $result = 0;

    switch($operation) {
        case "add":
            $result = $num1 + $num2;
            echo "<h3>Result: $num1 + $num2 = $result</h3>";
            break;
        case "subtract":
            $result = $num1 - $num2;
            echo "<h3>Result: $num1 - $num2 = $result</h3>";
            break;
        case "multiply":
            $result = $num1 * $num2;
            echo "<h3>Result: $num1 × $num2 = $result</h3>";
            break;
        case "divide":
            if($num2 != 0) {
                $result = $num1 / $num2;
                echo "<h3>Result: $num1 ÷ $num2 = $result</h3>";
            } else {
                echo "<h3>Error: Division by zero is not allowed!</h3>";
            }
```

```
        break;
      default:
        echo "<h3>Invalid Operation</h3>";
   }
}
?>
</body>
</html>
```

Input	Operation	Output
10, 5	add	10 + 5 = 15

(c) Briefly discuss sessions and cookies in PHP using appropriate example.

In web development, **maintaining user information across multiple pages** is a common requirement. PHP provides two primary mechanisms for this purpose: **cookies** and **sessions**. Both are used to **store and track information about a user**, but they differ in how and where the data is stored.

---

1. Cookies in PHP

Definition:
A **cookie** is a **small piece of data stored on the client's browser**. It allows a web application to **remember information about the user** across different visits or page requests. Cookies are sent automatically to the server with every HTTP request, which enables the server to **retrieve the stored data**.

Key Points:

- Cookies are stored **on the client-side**, inside the user's browser.
- They can have a **specific expiration time**, after which the browser automatically deletes them.
- If no expiration is set, the cookie lasts only for the **current browsing session**.
- Cookies are commonly used to **remember user preferences, login information, language settings, or shopping cart contents**.
- They are accessible in PHP through the **`$_COOKIE` superglobal array**.

Example: Setting and Reading a Cookie

```
<?php
// Setting a cookie named 'username' with value 'Komal' that expires in 1
hour
setcookie("username", "Komal", time() + 3600);

// Checking if the cookie exists
if(isset($_COOKIE["username"])) {
    echo "Welcome back, " . $_COOKIE["username"] . "!";
} else {
    echo "Hello! We have set a cookie for you.";
}
?>
```

Explanation:

- `setcookie()` creates a cookie. The first parameter is the **name**, the second is the **value**, and the third is the **expiration time** in seconds.
- `$_COOKIE["username"]` retrieves the value of the cookie if it exists.
- Cookies allow **data persistence on the client's machine**, which can be used even if the user closes and reopens the browser (depending on expiration).

---

2. Sessions in PHP

Definition:
A **session** is a mechanism to store information on the **server side**, which is associated with a **unique session ID**. The session ID is usually sent to the client in the form of a **cookie** so that the server can identify the user on subsequent requests. Sessions are more **secure than cookies** because sensitive information is **not stored on the client**.

Key Points:

- Sessions store data **on the server**, making them suitable for sensitive information like login credentials.
- Each user gets a **unique session ID**, which PHP uses to link the user to their session data.
- Session data is stored in the `$_SESSION` superglobal array.
- To start using a session, `session_start()` must be called at the **beginning of the PHP page** before any HTML output.
- Sessions can be destroyed using `session_destroy()` when the user logs out or when the data is no longer needed.

Example: Using PHP Sessions

```
<?php
session_start(); // Starting the session

// Storing data in session variables
$_SESSION["username"] = "Komal";
$_SESSION["role"] = "admin";
```

```
// Accessing session variables
echo "Welcome, " . $_SESSION["username"] . "!<br>";
echo "Your role is: " . $_SESSION["role"] . "<br>";

// Destroying the session (optional)
// session_destroy();
?>
```

How Cookies and Sessions Work Together

- When a session is started, PHP typically sends a **cookie containing the session ID** to the client.
- The client sends this cookie back to the server on subsequent requests, allowing PHP to **identify the user** and retrieve the associated session data.
- Cookies may store **non-sensitive or persistent user preferences**, while session data handles **secure and temporary information**.

Example: Session with Cookies Interaction

```
<?php
session_start();

// Set a cookie for user preference
setcookie("theme", "dark", time() + 86400); // expires in 1 day

// Set session variable for login
$_SESSION["username"] = "Komal";

echo "Hello " . $_SESSION["username"] . "!<br>";
if(isset($_COOKIE["theme"])) {
    echo "Your selected theme is: " . $_COOKIE["theme"];
}
?>
```

Output Example:

```
Hello Komal!
Your selected theme is: dark
```

Q.5 (a) Write a short note on file handling in PHP.

File handling in PHP is a fundamental feature that allows a web application to manage files on the server.

It enables developers to create, read, write, update, and delete files, which is useful for storing data persistently outside a database.

PHP provides several built-in functions to handle files efficiently, making it possible to manage user input, logs, reports, and other important data. Unlike variables, which store data temporarily, files allow information to **persist across multiple sessions** and can be accessed anytime by the application.

Files in PHP can be **opened in different modes**, such as read-only, write-only, append, or read-and-write.

Reading files can be done either line by line using `fgets()` or by reading the entire file content at once using `file_get_contents()`.

Writing to files is done using `fwrite()`, and it can either overwrite existing content or append new data depending on the mode used.

It is important to close a file after performing operations using `fclose()` to ensure that **all data is saved properly and server resources are freed**.

Files can be checked for existence using `file_exists()`, which prevents errors while reading or deleting files. Deleting a file is performed using `unlink()`, which removes the file permanently from the server.

File handling allows **dynamic interaction with users**, such as saving form inputs, logging user activities, generating reports, or maintaining configuration files for web applications. Security considerations are also important because improper handling may lead to unauthorized access or data loss.

PHP file handling can be used to store **structured or unstructured data**, including text, CSV, JSON, or even binary files like images.

It is versatile enough to support reading and writing in different formats, making it useful for small-scale applications or temporary storage before sending data to a database. Combining file handling with loops or conditional checks allows developers to **process large amounts of data efficiently**, such as reading logs line by line or appending multiple entries to a file.

Example

```
<?php
$file = fopen("example.txt", "w"); // Create or open file
fwrite($file, "This is a sample file.\n");
fwrite($file, "PHP file handling stores and processes data.\n");
fclose($file);

$file = fopen("example.txt", "r"); // Read file content
while(!feof($file)) {
    echo fgets($file) . "<br>";
}
fclose($file);
?>
```

Output:

```
This is a sample file.
PHP file handling stores and processes data.
```

Applications

- Storing **user data or form submissions** temporarily or permanently.
- Logging **errors, user activity, or transactions** for auditing.
- Reading **configuration files** to dynamically adjust web application settings.
- Generating **reports** in text or CSV format for users or administrators.
- Temporarily storing **data processing results** before inserting into a database.

(b) Write a note on AJAX.

AJAX stands for **Asynchronous JavaScript and XML**. It is a **technique used in web development** to create **interactive and dynamic web applications**. AJAX allows a web page to **send and receive data from a server asynchronously**, without having to reload the entire page. This makes web applications faster, smoother, and more responsive to user interactions. Despite the name, AJAX is not limited to XML; it can use **JSON, HTML, or plain text** as the data format.

Key Features of AJAX:

- **Asynchronous Communication:** Data can be exchanged with the server in the background, without interrupting the user's interaction with the page.
- **Partial Page Update:** Only the required part of the page is updated, reducing server load and improving performance.
- **Cross-Browser Compatibility:** AJAX works with all modern browsers using the `XMLHttpRequest` object or the newer `fetch()` API.
- **Dynamic User Experience:** Enables real-time updates, form validations, live search suggestions, and other interactive features.

How AJAX Works:

1. The **user triggers an event** (like clicking a button or typing in a search box).
2. JavaScript creates an **AJAX request** using `XMLHttpRequest` or `fetch()`.
3. The request is sent to the **server asynchronously**.
4. The server processes the request and sends back a **response** (data in JSON, XML, or HTML).
5. JavaScript updates the **web page dynamically** using the response without reloading the page.

Example Using AJAX (JSON Example):

```
<!DOCTYPE html>
<html>
<head>
    <title>AJAX Example</title>
    <script>
```

```
        function loadData() {
            var xhr = new XMLHttpRequest();
            xhr.onreadystatechange = function() {
                if(xhr.readyState == 4 && xhr.status == 200) {
                    document.getElementById("result").innerHTML =
xhr.responseText;
                }
            };
            xhr.open("GET", "data.txt", true); // Request data from server
            xhr.send();
        }
    </script>
</head>
<body>
    <h2>AJAX Example</h2>
    <button onclick="loadData()">Load Data</button>
    <div id="result"></div>
</body>
</html>
```

Applications of AJAX:

- Real-time **form validation** without page reload.
- **Live search suggestions** (like Google search).
- **Chat applications** for sending and receiving messages instantly.
- **Updating user dashboards or notifications** dynamically.
- Loading **data tables, stock prices, or weather updates** without refreshing the page.

(c) Design Employee form with Employee ID, Employee Name, Designation details with submit button using HTML. On clicking submit button, entered Employee details should be fetch and display on another webpage "EmpInfo.php".

1. Employee Form Page – `EmployeeForm.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Employee Form</title>
</head>
<body>
    <h2>Employee Registration Form</h2>
    <form action="EmpInfo.php" method="post">
        <label for="empid">Employee ID:</label>
        <input type="text" id="empid" name="empid" required><br><br>

        <label for="empname">Employee Name:</label>
        <input type="text" id="empname" name="empname" required><br><br>

        <label for="designation">Designation:</label>
```

```
        <input type="text" id="designation" name="designation"
required><br><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

2. PHP Page to Display Employee Info – `EmpInfo.php`

```
<?php
// Check if form data is received
if($_SERVER["REQUEST_METHOD"] == "POST") {
    $empID = $_POST['empid'];
    $empName = $_POST['empname'];
    $designation = $_POST['designation'];
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Employee Information</title>
</head>
<body>
    <h2>Employee Details</h2>
    <p><strong>Employee ID:</strong> <?php echo $empID; ?></p>
    <p><strong>Employee Name:</strong> <?php echo $empName; ?></p>
    <p><strong>Designation:</strong> <?php echo $designation; ?></p>
</body>
</html>
```

OR

Q.5 (a) Briefly discuss JSON.

JSON stands for **JavaScript Object Notation**. It is a **lightweight data interchange format** that is easy to read and write for humans, and easy to parse and generate for machines.

JSON is widely used in **web applications** to exchange data between a client (browser) and a server. It is simpler and less verbose than XML, making it faster to process.

Key Features of JSON:

- **Text-based format:** Can be transmitted over a network easily.
- **Language-independent:** Supported by almost all programming languages.
- **Structured data representation:** Uses **key-value pairs** for objects and arrays for lists of values.
- **Lightweight and efficient:** Minimal syntax reduces data size and improves performance.

JSON Structure:

1. **Object:** Enclosed in `{ }`, containing key-value pairs.
2. **Array:** Enclosed in `[ ]`, containing multiple values.
3. **Values:** Can be a string, number, boolean, array, object, or null.

Example of a JSON Object:

```
{
  "employeeID": "E101",
  "name": "John Doe",
  "designation": "Developer",
  "skills": ["JavaScript", "HTML", "CSS"]
}
```

Applications of JSON:

- Exchanging data between **client and server** in web applications.
- Representing **configuration settings** for applications.
- Widely used in **APIs** to send and receive structured data.
- Used in **mobile applications** for data communication with servers.
- Ideal for **storing structured information** in a readable format.

(b) Write the differences between synchronous and asynchronous web programming.

Feature	Synchronous Web Programming	Asynchronous Web Programming
Definition	Executes tasks **one after another**; the next task starts only after the previous one completes.	Executes tasks **independently**; tasks can run in the background without waiting for previous tasks to finish.
User Experience	Can cause the **browser to freeze or wait**, as user must wait for operations to complete.	Provides **smooth and responsive experience**, as operations run in the background.
Data Fetching	Server requests are **blocking**; user must wait until response is received.	Server requests are **non-blocking**; user can continue interacting with the page.
Example	Traditional form submission, page reloads.	AJAX requests, `fetch()` API, promises, and async/await in JavaScript.
Performance	Slower for multiple requests because each request waits for the previous one.	Faster and efficient for multiple requests; multiple operations can run simultaneously.

Feature	Synchronous Web Programming	Asynchronous Web Programming
Error Handling	Errors are handled **immediately**, stopping execution if an error occurs.	Errors can be handled asynchronously using **callbacks, promises, or try/catch with async/await**.
Use Cases	Simple scripts, sequential processing where order matters.	Modern web apps, real-time updates, data fetching without page reloads, chat applications.

(c) Discuss steps to connect database using PHP. Also, discuss any two database operations using proper example.

In web development, PHP is widely used to interact with databases such as **MySQL** or **MariaDB**. Database connectivity allows a web application to **store, retrieve, update, and delete data**, enabling dynamic content and persistent storage. PHP provides multiple ways to connect to a database, such as **MySQLi (MySQL Improved)** and **PDO (PHP Data Objects)**.

Steps to Connect to a Database Using PHP

1. **Select the Database System:**
   - Decide which database to use, most commonly MySQL or MariaDB. These databases are widely supported and integrate easily with PHP.
2. **Provide Connection Parameters:**
   - Hostname (e.g., `localhost`)
   - Database username (e.g., `root`)
   - Database password
   - Database name
3. **Establish the Connection:**
   - Using MySQLi procedural style:
   - `$conn = mysqli_connect($servername, $username, $password, $dbname);`
   - Using PDO (object-oriented, more secure and flexible):
   - `$conn = new PDO("mysql:host=$servername;dbname=$dbname", $username, $password);`
4. **Check the Connection:**
   - MySQLi: `mysqli_connect_error()` returns an error if the connection fails.
   - PDO: Use `try-catch` block to catch exceptions.
5. **Perform Database Operations:**
   - Execute **SQL queries** like `INSERT`, `SELECT`, `UPDATE`, `DELETE`.
   - PHP functions like `mysqli_query()` or PDO methods such as `prepare()` and `execute()` are used.
6. **Close the Connection:**
   - MySQLi: `mysqli_close($conn)`
   - PDO: Connection closes automatically when the object goes out of scope.

Example: Connecting to a Database

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "company";

// Create connection
$conn = mysqli_connect($servername, $username, $password, $dbname);

// Check connection
if(!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

Database Operations in PHP

1. Insert Operation

Definition:
The **Insert operation** is used to **add new records** into a database table. It is commonly used when a user submits a form and the data needs to be stored for future use.

Explanation:

- You prepare an **SQL query** using the `INSERT INTO` command, specifying the table and column values.
- Execute the query using PHP's `mysqli_query()` function.
- If the insertion is successful, the data is saved in the database; otherwise, an error message is returned.

Example:

```
<?php
$sql = "INSERT INTO employees (empID, name, designation) VALUES ('E101',
'Alice', 'Manager')";
if(mysqli_query($conn, $sql)) {
    echo "Record inserted successfully";
} else {
    echo "Error: " . mysqli_error($conn);
}
?>
```

---

2. Select (Fetch) Operation

Definition:
The **Select operation** is used to **retrieve records** from a database table. It allows displaying stored data on a webpage or using it in application logic.

Explanation:

- Prepare an **SQL query** using the `SELECT` command to specify the columns to fetch.
- Execute the query using `mysqli_query()`.
- Use `mysqli_fetch_assoc()` to retrieve each row as an associative array and process it as needed.

Example:

```
<?php
$sql = "SELECT empID, name, designation FROM employees";
$result = mysqli_query($conn, $sql);

while($row = mysqli_fetch_assoc($result)) {
    echo "ID: " . $row["empID"] . " - Name: " . $row["name"] . " -
Designation: " . $row["designation"] . "<br>";
}
?>
```